

Carp24 & Supabase

Administrator Playbook

Einfach verständlich: Was ist Supabase, was kann es, und wie verwaltest du Carp24 als Admin.

Version 1.0 | 21.08.2026 | Dokument-ID: EO-ADMIN-2026-CARP24-01-REV
einfach-online.dev | Philipp Schlemmer

Inhaltsverzeichnis

1	Changelog & Änderungsverzeichnis	3
2	Teil A — Supabase: Was ist das überhaupt?	4
2.1	Was ist Supabase?	4
2.2	Die Bausteine (Container) auf unserem Server	4
2.3	Kernkonzepte, die du als Admin verstehen solltest	4
2.4	Wie kommst du an die Datenbank?	5
3	Teil B — Carp24: Die Plattform	6
3.1	Was ist Carp24?	6
3.2	Alle Produkt-Features (Stand v1.0)	6
3.3	Das Admin-Dashboard (dein wichtigstes Werkzeug)	7
3.3.1	Übersicht	7
3.3.2	Reports (Meldungen moderieren)	7
3.3.3	Nutzer verwalten	7
3.3.4	Inhalte moderieren	7
3.3.5	Revenue (Abo-Übersicht)	7
3.3.6	Audit (Protokoll)	7
3.4	Die wichtigsten Datenbank-Tabellen	8
4	Teil C — Wo kann man was einstellen?	9
4.1	Konfigurationsdateien auf dem Server	9
4.2	Google-/Facebook-Login aktivieren (OAuth)	9
4.3	Stripe aktivieren (Pro-Abo)	10
4.4	KI-Assistent aktivieren	10
4.5	Web-Push-Erinnerung (täglicher Cron)	10
4.6	Pro-Status & Rollen manuell setzen	10
4.7	Wartungsmodus schalten [v1.0]	10

5	Teil D — Betrieb, Wartung & Fehler	12
5.1	Neue Version deployen	12
5.2	Migrationen einspielen	12
5.3	Backups & Wiederherstellung	12
5.4	Logs lesen	12
5.5	Typische Fehler und ihre Lösungen	13
5.6	Sicherheitsregeln (unbedingt)	13
5.7	Wichtige technische Merksätze	13
6	Glossar	14

1 Changelog & Änderungsverzeichnis

Version	Datum	Änderung
v1.1	21.08.2026	Wartungsmodus-Kapitel ergänzt (Schalten, Wirkung, Technik)
v1.0	21.08.2026	Initiales Dokument: Supabase-Grundlagen, Carp24-Features, Admin-Dashboard v2, Einstellungen, Betrieb & Fehlerbehandlung

Hinweis: Dieses Dokument enthält **keine Passwörter oder Secret-Werte**. Alle Schlüssel liegen im Passwort-/Secret-Handling des Orchestrators (Hermes) und in den lokalen `.env`-Dateien des Servers.

2 Teil A — Supabase: Was ist das überhaupt?

2.1 Was ist Supabase?

Supabase ist eine **Open-Source-Alternative zu Firebase** (Google). Es ist eine Komplet-Plattform für das Backend einer Web-App und besteht im Kern aus:

- **PostgreSQL-Datenbank** — eine der mächtigsten relationalen Datenbanken (SQL)
- **Authentifizierung (Auth)** — Registrierung, Login, Passwort-Reset, Social-Login (Google/Facebook), Zwei-Faktor (MFA)
- **Auto-API** — Jede Tabelle bekommt automatisch eine REST-API (PostgREST); kein Backend-Code nötig
- **Storage** — Datei-Upload (z.B. Fang-Fotos), S3-kompatibel
- **Realtime** — Live-Updates im Browser (WebSocket), z.B. für den Chat
- **Studio** — eine Weboberfläche, in der du Tabellen, Daten und Einstellungen direkt ansehen kannst

Supabase läuft bei uns **selbst gehostet** auf dem Server als Docker-Container — nicht in der Cloud von Supabase. Das heißt: Alle Daten bleiben auf unserer Infrastruktur (DSGVO-freundlich).

2.2 Die Bausteine (Container) auf unserem Server

Container	Aufgabe	Hinweis
supabase-db	PostgreSQL 17	Die eigentliche Datenbank (Port 5432 intern)
supabase-auth	GoTrue v2.189	Login/Registrierung/Passwort/MFA/JWT
supabase-rest	PostgREST v14	Auto-REST-API (Port 3000 intern)
supabase-storage	Storage v1.60.4	Datei-Upload (Fang-Fotos)
supabase-realtime	Realtime v2.102	Live-WebSockets (Chat)
supabase-kong	Kong 3.9.3	API-Gateway: zentrale Tür (extern 100.93.250.103:8055)
supabase-studio	Studio	Web-UI zur Datenbank-Verwaltung
supabase-meta	postgres-meta	Verwaltungs-API hinter Studio
supabase-pooler	Supavisor	Verbindungs-Pooling (Port 6543)

Wichtig für dich: Alle Wege nach außen laufen über **Kong** (Port 8055). Die Carp24-Web-App erreicht Supabase über den Apache-Proxy :8094, der auf Kong weiterleitet (/auth, /rest, /storage, /realtime).

2.3 Kernkonzepte, die du als Admin verstehen solltest

Tabellen Daten liegen in Tabellen (z.B. `catches` für Fänge). Jede Zeile = ein Datensatz.

RLS (Row Level Security) Eine Sicherheits-Ebene **in der Datenbank**: Legt fest, wer welche Zeilen sehen/ändern darf (z.B. „jeder nur seine eigenen Fänge“). Ohne RLS wäre alles für jeden erreichbar, der den API-Schlüssel kennt.

Policies Die konkreten RLS-Regeln pro Tabelle (z.B. `USING (user\_id = auth.uid())`).

Migrationen SQL-Dateien, die Schema-Änderungen dokumentieren (Ordner `supabase/migrations/`). Sie sind der „Stammbaum“ der Datenbank — niemals einfach so ändern, sondern neue Datei anlegen.

Buckets Speicher-Container für Dateien (bei Carp24: `catch-photos`).

Publicationen Listen von Tabellen, die Realtime-Updates senden dürfen (z.B. `chat\_messages`).

RPC-Funktionen Eigene SQL-Funktionen, die von der App aufgerufen werden (z.B. `import\_catches` für den CSV-Import).

JWT / Session Nach dem Login bekommt der Browser ein signiertes Token (JWT). Die API erkennt daran den Benutzer (`auth.uid()`).

2.4 Wie kommst du an die Datenbank?

Es gibt drei Wege:

1. **Supabase Studio** (komfortabel, Web-UI) — Tabellen, Zeilen, SQL-Editor
2. **psql** (Kommandozeile, direkt im Container) — für Migrationen und SQL
3. **REST-API** über Kong — was die App selbst nutzt

Migrationen einspielen (Standardweg, vom Orchestrator ausgeführt):

```
cd /mnt/projekte/carp24-fangbuch
PW=$(docker exec supabase-db printenv PGPASSWORD)
docker exec -i --env PGPASSWORD="$PW" supabase-db psql -h localhost -U
supabase_admin -d postgres < supabase/migrations/00XX_name.sql
```

3 Teil B — Carp24: Die Plattform

3.1 Was ist Carp24?

Carp24 ist eine **Karpfenangler-Community mit digitalem Fangbuch** (eigenes Produkt von einfach-online.dev). Angler erfassen ihre Fänge (Gewicht, Fischart, Gewässer, Foto, Wetter), sammeln Badges, sehen Jahres-Rückblicke und tauschen sich in Forum, Chat, Community-Board und Marktplatz aus. Es gibt ein Free-/Pro-Modell (Stripe) und einen KI-Angelassistenten.

Technik:

- Frontend: **Astro 7** (SSR, Node-Adapter), Code in `/mnt/projekte/carp24-fangbuch/web/`
- Backend: **Supabase self-hosted** (siehe Teil A)
- Dev-Lauf: **http://100.93.250.103:8094** (Apache → Node :4321)
- Deploy: systemd-Dienst `carp24-web` (`sudo systemctl restart carp24-web`)
- Branch: `sprint-1`, Migrationen 0013-0030

3.2 Alle Produkt-Features (Stand v1.0)

Bereich	Feature	Kurzbeschreibung
Kern	Fänge erfassen	Fang-Formular mit Foto, Gewicht, Art, Gewässer, Wetter, Koordinaten
Kern	Statistik	Diagramme & Auswertungen der eigenen Fänge
Kern	Offline-Modus	Fänge ohne Internet zwischenspeichern
A	CSV-Import	Alte Fanglisten als CSV importieren (max. 500 Zeilen)
A	Badges	10 Auszeichnungen (Erster Fang, 100 Kilo, Regenfischer ...)
A	Jahres-Rückblick	KPI-Karten, Monats-Balken, Mond-Statistik, Teilen
A	Trips	Angelausflüge anlegen und Fänge zuordnen
B	OAuth-Login	Google-/Facebook-Buttons (Creds konfigurierbar)
C	Community-Board	Öffentliche Top-Fänge teilen (Public-Toggle)
C	Forum	Threads + Antworten mit 4 Kategorien, Melden, Moderation
C	Chat	Realtime-Chat mit Live-WebSocket, Melden, Löschen
D	Pro-Gating	Free vs. Pro (Locks), Admin sieht alles
D	Stripe-Abo	Checkout + Webhook setzt <code>is_pro</code> (Test-Modus)
E	i18n	Deutsch/Englisch-Umschalter, Rechtstexte zweisprachig
E	Web-Push	Benachrichtigungen (VAPID), Antwort-Push, Tages-Erinnerung
E	KI-Assistent	Chat mit OpenRouter-API, keine Speicherung (DSGVO)
E	Marktplatz	Anzeigen (Foto, Preis), In-App-Kontakt-Nachrichten
X	DSGVO-Export	Eigene Daten exportieren (Art. 15)
X	DSGVO-Löschung	Eigene Daten löschen (Art. 17)
X	Meldungen	<code>content_reports</code> — User melden Inhalte (DSA)

3.3 Das Admin-Dashboard (dein wichtigstes Werkzeug)

Erreichbar über `http://100.93.250.103:8094/admin/` nach Login mit einem Konto mit Rolle `ADMIN` oder `MODERATOR` (Header-Link **ADMIN**). Sechs Bereiche:

3.3.1 Übersicht

Zahlen auf einen Blick: Nutzer, Fänge, offene Meldungen, Threads, Chat-Nachrichten, Anzeigen, Trips, aktive Pro-User, Gebannte. Dazu ein 14-Tage-Wachstums-Diagramm (Nutzer/Fänge) und eine Integrations-Status-Karte (welche Dienste konfiguriert sind: Stripe, KI, Push, Service-Role — nur Ja/Nein, keine Schlüssel).

3.3.2 Reports (Meldungen moderieren)

Nutzer können Inhalte melden (Fang, Forum-Thread, Forum-Antwort, Chat-Nachricht, Marktplatz-Anzeige). Hier siehst du:

- Offene/erledigte Meldungen (Tabs), mit Grund, Melder und Ziel-Inhalt im Kontext
- Button **ERLEDIGT** — Meldung abschließen
- Button **INHALT LÖSCHEN** — Ziel-Inhalt entfernen und Meldung erledigen
- Button **BANNEN** — wenn ein User mehrfach auffällig ist (☐3 offene Meldungen)

3.3.3 Nutzer verwalten

- Suche (Name/E-Mail) + Filter (Rolle, Status, Pro)
- **Rolle** ändern: USER / MODERATOR / ADMIN
- **Pro** an/aus (ohne Stripe-Zahlung manuell setzen)
- **BANNEN / ENTBANNEN** (mit Grund) — blockiert auch den Login (GoTrue-Ban)
- **PASSWORT ZURÜCKSETZEN** — erzeugt ein temporäres Passwort (Support-Fall)
- **DETAIL** — öffnet den User-Feed: alle Fänge, Forum-Beiträge, Chat-Nachrichten, Anzeigen, Trips, Badges und Meldungen des Users auf einen Blick; Inhalte direkt löschar

3.3.4 Inhalte moderieren

Tabs für Fänge, Forum, Chat und Marktplatz. Durchsuchen und einzelne Inhalte löschen (Fänge werden „weich“ gelöscht / soft-delete, Rest hart).

3.3.5 Revenue (Abo-Übersicht)

Aktive Pro-User, Free-User, Pro-Quote, Liste der Pro-User, Stripe-Events-Status. Hinweis, wenn Stripe noch nicht konfiguriert ist.

3.3.6 Audit (Protokoll)

Wer hat wann was getan: Rollen-Änderungen, Banns, Passwort-Resets, Report-Bearbeitung, Inhalts-Löschung. Wichtig für die DSGVO-Nachweispflicht (Accountability).

3.4 Die wichtigsten Datenbank-Tabellen

Tabelle	Migration	Inhalt
profiles	Basis	Nutzerprofile (display_name, role, is_pro, banned_at, ...)
catches	Basis	Fänge (species, weight_kg, water_name, photo, is_public)
trips	0019	Angelausflüge (catches.trip_id verknüpft)
badges / user_badges	0018	10 Badges + Verleihungen
forum_threads / forum_posts	0021	Forum (category ab 0028)
chat_messages	0022	Realtime-Chat
content_reports	0017	Meldungen (status open/resolved)
marketplace_items	0026	Marktplatz-Anzeigen
marketplace_contacts	0029	In-App-Kontakt-Nachrichten
push_subscriptions	0025	Push-Abos der Browser
stripe_events	0024	Stripe-Webhook-Ereignisse (Idempotenz)
admin_audit_log	0030	Admin-Aktionen (Audit)

4 Teil C — Wo kann man was einstellen?

4.1 Konfigurationsdateien auf dem Server

1. `/mnt/projekte/carp24-fangbuch/web/.env` — Einstellungen der Web-App:

Variable	Wofür?	Status (21.08.)
PUBLIC_SUPABASE_URL	Basis-URL zu Supabase (über Apache :8094)	gesetzt
PUBLIC_SUPABASE_ANON_KEY	Öffentlicher API-Schlüssel (Browser)	gesetzt
SUPABASE_SERVICE_ROLE_KEY	Admin-Schlüssel (nur Server!)	gesetzt
STRIPE_SECRET_KEY	Stripe-Test-Server-Schlüssel	wartet auf Philipp
STRIPE_WEBHOOK_SECRET	Stripe-Webhook-Signatur	wartet auf Philipp
STRIPE_PRICE_ID	Stripe-Preis-ID für Pro-Abo	wartet auf Philipp
OPENROUTER_API_KEY	KI-Assistent (Open-Router)	wartet auf Philipp
VAPID_PUBLIC_KEY / VAPID_PRIVATE_KEY / VAPID_SUBJECT CRON_KEY	Web-Push Schutz für den Erinnerungs-Cron	gesetzt noch zu setzen

2. `/mnt/projekte/carp24-fangbuch/infra/.env` — **Supabase-Stack (nur Orchestrator ändern)**: Enthält JWT-Secret, ANON_KEY, SERVICE_ROLE_KEY, API-URLs und die GoTrue-Provider-Platzhalter für Google/Facebook-Login (`GOTRUE_EXTERNAL_GOOGLE_ENABLED` etc.).

3. **Supabase Studio** — für Tabellen, Zeilen, SQL und RLS direkt. Nicht öffentlich erreichbar; Zugriff über Docker/SSH.

4.2 Google-/Facebook-Login aktivieren (OAuth)

1. In Google Cloud Console / Facebook Developers eine OAuth-App anlegen

2. Redirect-URL: `http://100.93.250.103:8094/auth/v1/callback`

3. Client-ID + Secret in `infra/.env` eintragen (`GOTRUE\_EXTERNAL\_GOOGLE\_CLIENT\_ID` , `\_\_SECRET` , FB analog)
4. Auth-Container neu starten: `docker compose restart supabase-auth`
5. Test: Login-Seite → „Mit Google anmelden“

4.3 Stripe aktivieren (Pro-Abo)

1. Stripe-Konto (Test-Modus) → API-Keys: `STRIPE\_SECRET\_KEY` (sk_test...) in `web/.env`
2. Produkt + Preis anlegen → `STRIPE\_PRICE\_ID`
3. Webhook einrichten: URL `https://<domain>/api/stripe/webhook` , Event `checkout.session.completed` + `customer.subscription.deleted/updated` → `STRIPE\_WEBHOOK\_SECRET`
4. `SUPABASE\_SERVICE\_ROLE\_KEY` muss gesetzt sein (ist es)
5. Test: Premium-Seite → „JETZT UPGRADEN“ → Test-Checkout → `is\_pro=true`

4.4 KI-Assistent aktivieren

OpenRouter-Key (`OPENROUTER\_API\_KEY`) in `web/.env` setzen. Ohne Key liefert der Assistent sauber einen 503-Hinweis (Feature ist dann deaktiviert statt kaputt).

4.5 Web-Push-Erinnerung (täglicher Cron)

Die API-Route `/api/push/daily-reminder` wird per systemd-Timer (täglich 08:00) aufgerufen und braucht den Header `x-cron-key` . Vorher `CRON\_KEY` in `web/.env` setzen, dann richtet der Orchestrator den Timer ein.

4.6 Pro-Status & Rollen manuell setzen

Im Admin-Dashboard unter **Nutzer**: Rolle-Select (USER/MODERATOR/ADMIN) und Pro-Checkbox. Alternativ direkt in der Datenbank:

```
docker exec --env PGPASSWORD="$PW" supabase-db psql -h localhost -U
supabase_admin -d postgres \
-c "UPDATE profiles SET role='ADMIN', is_pro=true WHERE
id='<user-id>';"
```

4.7 Wartungsmodus schalten [v1.0]

Bei Wartungsarbeiten kannst du die gesamte Plattform auf eine Wartungsseite stellen:

1. Admin-Dashboard → **Übersicht** → Karte **WARTUNGSMODUS**
2. Optional: Nachricht an Besucher eingeben (z.B. „Wir sind bald wieder da!“)
3. Button **WARTUNGSMODUS AKTIVIEREN** — sofort aktiv

Wirkung: Alle Besucher (nicht eingeloggt oder ohne Admin-Rolle) sehen `/wartung` (HTTP 503). **Admins** (Rolle ADMIN/MODERATOR) kommen überall durch — auch `/admin/*`. `/login`, `/impressum` und `/datenschutz` bleiben erreichbar (damit du dich als Admin einloggen kannst und Rechtstexte zugänglich bleiben). Zum Ausschalten: Button **WARTUNGSMODUS DEAKTIVIEREN** (gleiche Karte). **Technik:** Der Status liegt in der Tabelle `app\_settings` (Key `maintenance`, Migration 0031) — überlebt Server-Neustarts. Ein globaler Astro-Middleware-Guard prüft jeden Request (5-Sekunden-Cache). Schalt-Aktionen landen im Audit-Log (`settings.maintenance`).

5 Teil D — Betrieb, Wartung & Fehler

5.1 Neue Version deployen

1. Code ändern lassen (OpenCode-Briefings, nie direkt am Server)
2. Build: `cd /mnt/projekte/carp24-fangbuch/web \&\& npm run build`
3. Neustart: `sudo systemctl restart carp24-web`
4. Prüfen: `systemctl is-active carp24-web` `active` ; Seite per Browser testen

Hinweis: Während eines Builds kann der laufende Server kurz 500er liefern (dist wird überschrieben). Nach dem Restart ist alles wieder normal. Nicht während eines Builds testen.

5.2 Migrationen einspielen

Neue SQL-Datei liegt in `supabase/migrations/`. Einspielen siehe Teil A (psql-Kommando). Nach FK-/Schema-Änderungen ggf. `docker compose restart supabase-rest` (Schema-Reload).

5.3 Backups & Wiederherstellung

Backup-Lauf täglich 02:00 (Docker-Dumps). Restore-Drill dokumentiert im Vault (RESTORE.md). Vor größeren Änderungen: Backup prüfen.

5.4 Logs lesen

```
sudo journalctl -u carp24-web --no-pager -n 100          # Web-App-Log
sudo journalctl -u carp24-web --since "10 min ago"      # aktuelle Fehler
docker logs supabase-auth --tail 50                    # Auth-Log
```

5.5 Typische Fehler und ihre Lösungen

Symptom	Ursache / Fix
Fangliste zeigt „keine Fänge“	Null-Filter falsch: <code>IMMER</code> verwenden, nie <code>.is("deleted_at", null)</code> verwenden, nie <code>.eq(..., null)</code>
500 „supabaseKey is required“	<code>SERVICE_ROLE_KEY</code> in <code>web/.env</code> fehlt oder Build lief vor dem Setzen □ Key setzen + neu bauen
Forum-Seite 500 „t2 is not a function“	i18n-Überschattung: In <code>map</code> -Callbacks nie den Parameter <code>t</code> nennen
Foto-Upload schlägt fehl	Storage-Bucket <code>catch-photos</code> + <code>GLOBAL\S3\BUCKET</code> muss gesetzt sein
Login/Realtime funktioniert nicht	Kong muss auf erreichbarer IP binden; Apache-Proxy <code>/auth</code> <code>/realtime</code> mit WebSocket-Upgrade
Seite crasht client-seitig	PageErrors im Browser prüfen; anonyme Besucher müssen auf <code>/login</code> redirectet werden (SSR-Guard)

5.6 Sicherheitsregeln (unbedingt)

- **Niemals Secrets ins Git-Repo** — `.env`-Dateien sind ausgeschlossen (gitignore). Secret-Scan blockt automatisch
- `SUPABASE\SERVICE\ROLE\KEY` nur auf dem Server (nie im Browser-Code)
- RLS ist die Sicherheitsgrenze — Policies nie ohne Review ändern
- Admin-Aktionen landen im `admin\_audit\_log` (DSGVO)
- Benachrichtigungs-/E-Mail-Tests nur mit echten Adressen (keine Fake-Domains)

5.7 Wichtige technische Merksätze

- DB heißt `postgres`; User `supabase\_auth\_admin` für Auth-Daten, `supabase\_admin` für DDL
- Supabase-Stack NICHT einzeln pinnen — gesamten Stack auf einem Release-Stand halten (Doku-Regel MASTER_SPRINT)
- Realtime-Chat braucht die Tabelle in der Publication `supabase\_realtime`
- Temp-Watchdog (systemd) räumt hängende Test-Browser automatisch auf

6 Glossar

Admin-Dashboard Die interne Verwaltungsoberfläche von Carp24 (`/admin/`)

Badge Spielerische Auszeichnung im Profil (10 Stück)

DSGVO Datenschutz-Grundverordnung (EU); Art. 15/17 = Auskunft/Löschung

DSA Digital Services Act (EU); Meldepflichten für User-Content

GoTrue Supabase-Auth-Dienst (Login, JWT, MFA)

is_pro Flag auf dem Profil: zahlender Pro-User

JWT Signiertes Login-Token (JSON Web Token)

Migration Versionierte SQL-Änderung an der Datenbank

PostgREST Werkzeug, das aus SQL-Tabellen automatisch eine REST-API macht

RLS Row Level Security: Zeilen-Sicherheit in der Datenbank

RPC Eigene SQL-Funktion, die die App aufruft

VAPID Schlüsselpaar für Web-Push-Benachrichtigungen